

sonoisa / **sentence-bert-base-ja-mean-tokens-v2** like 49

Feature Extraction sentence-transformers PyTorch Safetensors

Japanese bert sentence-bert sentence-similarity License: cc-by-sa-4.0

Train Deploy Use this model

Model card Files xet Community 2

Downloads last month
48,290

Safetensors

Model size 11

Inference Providers NEW

Feature Extraction

Your sentence here...

View Code

Spaces using sonoisa/sentence-bert-base-ja-mean-tokens-v2 5

Run 15,000+ Models Instantly

Inference Providers let you run inference on thousands of models served by our partners using a simple, unified, OpenAI-compatible serverless API ([Learn more](#)).

sonoisa/sentence-bert-base-ja-mean-tokens-v2 is supported by the following Inference Providers:

HF Inference API

View API Code

Dismiss

This is a Japanese sentence-BERT model.

日本語用Sentence-BERTモデル（バージョン2）です。

バージョン1よりも良いロス関数である MultipleNegativesRankingLoss を用いて学習した改良版です。
手元の非公開データセットでは、バージョン1よりも1.5～2ポイントほど精度が高い結果が得られました。

事前学習済みモデルとして cl-tohoku/bert-base-japanese-whole-word-masking を利用しました。
従って、推論の実行には fugashi と ipadic が必要です（`pip install fugashi ipadic`）。

旧バージョンの解説

<https://qiita.com/sonoisa/items/1df94d0a98cd4f209051>

モデル名を "sonoisa/sentence-bert-base-ja-mean-tokens-v2" に書き換えれば、本モデルを利用した挙動になります。

使い方

```
from transformers import BertJapaneseTokenizer, BertModel
import torch
```

```

class SentenceBertJapanese:
    def __init__(self, model_name_or_path, device=None):
        self.tokenizer = BertJapaneseTokenizer.from_pretrained(model_name_or_path)
        self.model = BertModel.from_pretrained(model_name_or_path)
        self.model.eval()

        if device is None:
            device = "cuda" if torch.cuda.is_available() else "cpu"
        self.device = torch.device(device)
        self.model.to(device)

    def _mean_pooling(self, model_output, attention_mask):
        token_embeddings = model_output[0] #First element of model_output contains all token embeddings
        input_mask_expanded = attention_mask.unsqueeze(-1).expand(token_embeddings.size()).float()
        return torch.sum(token_embeddings * input_mask_expanded, 1) / torch.clamp(input_mask_expanded.sum(1), min=1e-9, max=1e9)

    @torch.no_grad()
    def encode(self, sentences, batch_size=8):
        all_embeddings = []
        iterator = range(0, len(sentences), batch_size)
        for batch_idx in iterator:
            batch = sentences[batch_idx:batch_idx + batch_size]

            encoded_input = self.tokenizer.batch_encode_plus(batch, padding="longest",
                                                             truncation=True, return_tensors="pt").to(self.device)
            model_output = self.model(**encoded_input)
            sentence_embeddings = self._mean_pooling(model_output, encoded_input["attention_mask"])

            all_embeddings.extend(sentence_embeddings)

        # return torch.stack(all_embeddings).numpy()
        return torch.stack(all_embeddings)

MODEL_NAME = "sonoisa/sentence-bert-base-ja-mean-tokens-v2" # <- v2です。
model = SentenceBertJapanese(MODEL_NAME)

sentences = ["暴走したAI", "暴走した人工知能"]
sentence_embeddings = model.encode(sentences, batch_size=8)

print("Sentence embeddings:", sentence_embeddings)

```

